Chandubhai S Patel Institute of Technology

CHAROTAR UNIVERSITY OF SCIENCE AND TECHNOLOGY
FACULTY OF TECHNOLOGY AND ENGINEERING
B. TECH (IT/CE/CSE/AIML)

DEPSTAR

ADVANCED WEB DEVELOPMENT FRAMEWORKS (ITUE301)

Practical 1: Introduction to React and Component Architecture	
CO/PO	CO1 / PO3, PO5
Objective	To set up a React development environment using Vite and build a static UI using independently structured, reusable components.
Industry Relevance	Mirrors the standard local dev setup used in modern frontend teams Vite-based scaffolding and component reuse are foundational skills expected from day one in any React role.
Prerequisites	• Basic HTML and CSS • Familiarity with JavaScript fundamentals (variables, functions) • Node.js and npm installed on the lab machine
Theory Concepts (Self-Study Reference)	
Concepts to Review	• What React is and why component-based UI development is used over plain HTML/JS • JSX syntax embedding JavaScript expressions inside markup • Functional components and the role of the return statement • Props passing data from a parent component into a child component • Why reusable components reduce duplication and improve maintainability
Architecture / Diagram	
Component Tree	App.jsx ├── Header.jsx (site title, name) ├── About.jsx (short bio text) ├── Skills.jsx (list of skills, receives props) └── Footer.jsx (contact / copyright info)
Problem Definition	
Problem Statement	• Create a React application using Vite for a student portfolio page. • The application must include at least 4 reusable components: Header, About, Skills, and Footer. • Each component must be independently structured and composed into a single-page layout. • No logic or JSX should be duplicated across components. • Props must be used to pass at least one piece of data into a minimum of 2 components.
Key Questions / Analysis	• What is the role of each component in the overall UI structure? • How does React re-render when props change? • Why is component reusability important in large-scale applications?
Supplementary Problems	• Add a NavBar component that highlights the active section. • Pass an array of skills as a prop to the Skills component and render them dynamically. • Add a theme color prop to the Header and apply it as an inline style.
Key Skills Addressed	• Setting up a React project using Vite • Designing and structuring reusable functional components • Passing data between components using props • Composing a single-page layout from multiple components

Learning Outcome	Students will be able to set up a React development environment, create reusable components, and compose them into a working UI using props for data flow.

Lab Session Step-by-Step

Steps

1. Verify Node.js and npm are installed:

```
node -v
npm -v
```

2. Scaffold the React app using Vite:

```
npm create vite@latest student-portfolio -- --template react
cd student-portfolio
npm install
npm run dev
```

3. Inside src/components, create four files: Header.jsx, About.jsx, Skills.jsx, Footer.jsx.
4. Build the Skills component to accept a props array and render it as a list:

```
function Skills({ skillList }) {
  return (
    <ul>
      {skillList.map((s) => <li key={s}>{s}</li>)}
    </ul>
  );
}
```

5. Import and compose all four components inside App.jsx in a single layout.
6. Pass a name prop to Header and a skillList prop to Skills from App.jsx.
7. Run the app and verify all components render correctly without console errors.

Tools / Technology / Resources Required	• Node.js (v18+), npm • Visual Studio Code • Vite, React 18+ • Browser DevTools (Console + React DevTools extension)
Total Hours	• Lab Session: 2 hours • Implementation: 1.5 hours • Testing / Modification: 0.5 hours • Total Engagement: 2 hours

Faculty Teaching Guide

Guidance Notes	• Pre-check that lab machines have Node.js installed; keep an offline installer ready as backup. • Live-demo the Vite scaffold once, slowly, before letting students repeat it independently. • Walk the room during npm install slow lab Wi-Fi or proxy settings are the most common blocker. • Stress 'one component, one responsibility' while reviewing student component structure.
Common Mistakes	• Using an outdated Node version, causing the Vite scaffold command to fail. • Putting all UI code inside App.jsx instead of splitting into separate components. • Hardcoding values instead of passing them as props. • Forgetting to save a file before checking the browser, leading to confusion about why changes aren't visible.

Troubleshooting Guide

Symptom	Likely Cause	Fix

npm create vite hangs	Network/proxy blocked	Check lab internet connection; retry on stable network or use offline cache
Port 5173 already in use	Old dev server still running	Stop the previous process or run npm run dev -- --port 5174
Component not visible in browser	Component not imported/composed in App.jsx	Confirm import statement and that the component is placed inside the return block
Props show as undefined	Prop name mismatch between parent and child	Verify the prop name passed in App.jsx matches the name used in the child component

Student Assignment / Post Laboratory Work

Tasks	• Review the Theory Concepts section and the assigned Coursera module before next week. • Add a Projects component to the portfolio that renders a hardcoded list of 3 projects. • Experiment with passing different prop values and observe how the rendered UI changes.

GitHub Deliverables

Expected Repository Contents	• Public repository named portfolio-<rollno> • Working Vite + React project with 4+ components • Clean .gitignore (node_modules excluded) • At least one meaningful commit message describing the work

Rubrics (Practical Evaluation / Viva)

Criteria (Marks)	Description
Conceptual understanding (2 Marks)	Explains what and why, not just what was done
Implementation (4 Marks)	Code written, runs, and is original
Correctness (2 Marks)	Output matches expected result
Reflection (1 marks)	Errors documented and explained
Lab file evidence (1 marks)	Completeness and presentation
Total: 10 marks	**Passing Threshold: 5 / 10 (50%)**